Ders 17 Genişletilmiş Notu: Dinamik Programlamaya Giriş: Fibonacci, Para Sırası ve Sırt Çantası Problemleri

Ders 17 Genişletilmiş Notu: Dinamik Programlamaya Giriş: Fibonacci, Para Sırası ve Sırt Çantası Problemleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ara Adımdan Sıralama Algoritmasını Tanımak
- 2. Dinamik Programlama Nedir?
- 3. Optimizasyon ve Kombinatorik Problemler
- 4. Optimallik İlkesi
- 5. Fibonacci Sayıları
- 6. Para Sırası Problemi (Coin-Row Problem)
- 7. Sırt Çantası Problemi (0/1 Knapsack Problem)
- 8. Örnek Sorular
- 9. Dersin Toparlaması

1. Ara Adımdan Sıralama Algoritmasını Tanımak

Ders, vize haftasından sonraki ilk dersti. Asıl konuya geçmeden önce vizeye dair geri bildirimler konuşuldu. Bunların çoğu idari olduğu için bu nota alınmadı. Yalnız akademik bir nokta burada kayda değer.

Vizede, bir dizinin ara durumuna bakıp hangi sıralama algoritmasının çalıştığını bulma sorusu vardı. Bir öğrenci, sayıların aralıkları büyük olduğu için dizinin sıralı kısmını gözle bulmanın zorlaştığını söyledi. Hoca, bu soruda algoritmalara özel ipuçları olduğunu belirtti:

- **Insertion sort:** Elemanın sıralı kısımda kendi yerine yerleşip yerleşmediğine bakılır.
- **Merge sort:** Dizinin ikiye bölündüğü noktanın iki tarafının kendi içinde sıralı olup olmadığına bakılır.

Hocaya göre, algoritmalara bu özel bakış açısıyla bakıldığında soru çok karmaşık değildir. Ama gözle hızlı tarama yeteneği de gerektirir.

2. Dinamik Programlama Nedir?

Hoca konuya, daha önce dinamik programlamayı (dynamic programming) duyan olup olmadığını sorarak başladı. Bir öğrenci en küçük düzenleme uzaklığı (minimum edit distance) probleminin dinamik programlamayla çözüldüğünü bildiğini söyledi. Öğrencinin tarifi şuydu: Problem alt problemlere bölünür, her birinin çözümü bellekte tutulur, sonra bu çözümler ana probleme aktarılır. Hoca bunu “gayet güzel bir özet” olarak kabul etti.

Hocanın tanımı şöyledir: Dinamik programlamada bir problemin özellikle **örtüşen alt problemleri** (overlapping subproblems) çözülür ve sonuçları bellekte saklanır. Genellikle aşağıdan yukarıya (bottom-up) bir yaklaşım kullanılır. Küçük problemlerden büyük problemin çözümüne doğru, değerler saklanarak ilerlenir.

Bu yöntem 1950'lerde Richard Bellman tarafından ortaya atıldı. Adındaki "programming" sözcüğü burada bilgisayar programı yazmak anlamında değil, **planlama** (planning) anlamındadır.

Hoca yöntemin neden akıllıca olduğunu şöyle açıkladı: Önce alt problemleri çözüp değerlerini saklarsanız, ana problemin çözümüne ulaşırken özyinelemeli (recursive) yaklaşımda tekrar tekrar çözülen noktaları gereksiz yere yeniden çözmezsiniz. Bazı problemlerde özyinelemeli yaklaşım avantajlı değildir. Dinamik programlama bu dezavantajı gidermeyi amaçlar. Bunun nasıl olduğu Fibonacci örneğinde gösterildi.

Hoca ardından dinamik programlamanın tanımını ekrana yazdı ve öğrencilerden not almalarını istedi.

> **Ek bilgi (derste anlatılmadı):** Dinamik programlamanın iki yaygın uygulama biçimi vardır. Yukarıdan aşağıya (top-down) biçimde özyinelemeli fonksiyon korunur, ama hesaplanan her değer bir tabloya yazılır ve tekrar istendiğinde tablodan okunur. Buna memoization denir. Aşağıdan yukarıya (bottom-up) biçimde ise tablo en küçük alt problemden başlanarak döngüyle doldurulur. Bu derste hocanın kullandığı biçim ikincisidir.

3. Optimizasyon ve Kombinatorik Problemler

Hoca, dinamik programlamanın birçok problemde kullanılabildiğini, ama özellikle iki tür problemde kullanıldığını söyledi: optimizasyon problemleri ve kombinatorik problemler.

3.1 Optimizasyon Problemi Ne Değildir?

Hoca bir optimizasyon problemi örneği istedi. İlk cevaplar "verilen bir problemi daha hızlı ve daha az yer kaplayacak şekilde çözmek" ve "hantal çalışan bir programı optimize etmek" yönündeydi. Hoca burada bir yanlış algı olduğunu söyledi:

> **Hocanın vurgusu:** Optimizasyon problemi dendiğinde bir algoritmayı optimize etmekten, yani hızlandırmaktan bahsetmiyoruz. Optimizasyon problemlerini en küçük ya da en büyük değeri bulma problemleri olarak düşünebilirsiniz.

3.2 Optimizasyon Problemi Örnekleri

Hocaya göre optimizasyon problemleri özellikle Endüstri Mühendisliğinin sıkça ilgilendiği konulardır. Derste geçen örnekler şunlardır:

- **Graf boyama:** Bir grafı en az sayıda renkle boyama problemi (hocanın örneği).
- **Otobüs güzergâhı optimizasyonu:** Otobüslerin yollarını en iyi hâle getirmek (bir öğrencinin örneği).
- **Ders programı yerleştirme (class scheduling):** Üniversitede hangi dersin hangi sınıfta işleneceğine karar vermek (bir öğrencinin örneği). Hoca bunu "en az sayıda hoca ile en az saati kaplayarak derslerin yerleştirilmesi" diye somutlaştırdı.
- **Bütçeyle alışveriş:** Elinizde belli bir miktar para var. Bu parayla, fiyat-performans açısından size en fazla katkı sağlayacak malzemeleri almak istiyorsunuz (hocanın örneği).

Hocanın genel tanımı şöyledir: Elinizdeki kısıtlı kaynakları, size verilen kısıtlara uygun şekilde, en iyi biçimde değerlendirerek ürettiğiniz çözümler literatürde optimizasyon problemi olarak geçer.

> **Ek bilgi (derste anlatılmadı):** Grafı en az renkle boyama problemi NP-zor (NP-hard) bir problemdir. Genel hâli için polinom zamanlı bir algoritma bilinmemektedir. Bu yüzden derste bu örnek, dinamik programlamayla verimli çözülen bir problem olarak değil, yalnızca "optimizasyon problemi nedir" sorusunun bir örneği olarak verilmiştir.

3.3 Kombinatorik Problemler

Hoca kombinatorik problemleri bir sınıflandırma (classification) örneğiyle anlattı. Bir sınıflandırma probleminde bir örneğin hangi sınıfa ait olduğunu anlamak için özellikler (feature) kullanılır. Elimizde n özellik olsun. Bu n özelliğin hepsi birlikte örneğin %90 başarı sağlıyor.

Ama özelliklerden bazıları başarıyı kötü etkiliyor olabilir. İçlerinden belli başlılarını seçip kullanırsak başarı belki %95'e çıkar. Soru şudur: En iyi başarıyı veren alt kümeyi bulmak için kaç farklı seçeneğe bakmamız gerekir?

Bir öğrenci önce $n!$ dedi; bu yanlıştı. Başka bir öğrenci 2^n önerdi. Hoca cevabı $2^n - 1$ olarak düzeltti ve "neden eksi 1?" diye sordu. Öğrenci "kümenin tamamını kullanmayacağız" diye düşündü; hoca "tam tersi" dedi. Çıkarılan seçenek **boş kümedir**: Hiç özellik kullanmadan sınıflandırma yapılamaz.

$$\text{Boş olmayan alt küme sayısı} = 2^n - 1$$

Hocanın vurgusu: Bu, test sorusu olarak sorulabilecek çok basit bir sorudur.

4. Optimallik İlkesi

Tanımdan sonra hoca bir kavramın altını çizdi: **optimallik ilkesi** (principle of optimality).

Bu ilkeye göre bir problemin optimal sonucuna ulaşmak için küçük problemlerin optimal sonuçlarını bulmak bizi genelde doğru sonuca götürür. Bazı istisnalar (exception) dışında, alt problemlerin optimal sonuçları birleştirilerek ana problemin optimal sonucuna ulaşılabilir. Dinamik programlamanın birçok optimizasyon problemini çözebilmesini sağlayan da bu ilkedir.

Hoca bu ilkenin her problem için geçerli olmadığını, bunu ileride yeri geldiğinde tartışacaklarını söyledi.

5. Fibonacci Sayıları

Hoca, her sene dinamik programlamaya Fibonacci örneğiyle başladıklarını, çünkü konuyu çok güzel anlattığını söyledi. Dizi şöyle devam eder:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34,\ \ldots$$

5.1 Rekürans Bağıntısı

Hoca dizinin rekürans bağıntısını (recurrence relation) sordu. Bir öğrencinin cevabı $F(n-1) + F(n-2)$ oldu. Hoca buna iki başlangıç koşulunu (initial condition) ekledi:

$$F(n) = F(n-1) + F(n-2), \quad n > 1, \qquad F(0) = 0, \quad F(1) = 1$$

5.2 Özyinelemeli Çözüm

Hoca bağıntıyı doğrudan özyinelemeli bir fonksiyona çevirdi. Fonksiyona kaçıncı Fibonacci sayısını istediğimiz verilir:

```
int fibonacci(int n) {
    if (n == 0) return 0;   /* başlangıç koşulu */
    if (n == 1) return 1;   /* başlangıç koşulu */
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Hoca bu çözümde bir yanlışlık olmadığını, doğru sonucu üreteceğini, ama bir problemi olduğunu söyledi ve problemin ne olduğunu sordu.

Soru: Bu özyinelemeli çözümün sıkıntısı nedir? **Cevap:** Bir öğrenci önce "her adımda fonksiyonu iki kez çağırıyoruz, çağrı sayısı her adımda iki katına çıkıyor" dedi. Hoca bunun doğru olduğunu, ama asıl sorunun iki kez çağırmanın sonucu olduğunu söyledi. Öğrencinin ikinci cevabı asıl noktayı yakaladı: $F(n-1)$ hesaplanırken $F(n-2)$ de yeniden hesaplanır. Yani **daha önce hesaplanmış değerler tekrar tekrar hesaplanır.**

Hoca bunu çağrı ağacı üzerinde gösterdi. Bir dalda hesaplanan $F(2)$ başka bir dalda yeniden hesaplanır, aynı şey $F(3)$ için de geçerlidir. Sayılar büyüdükçe bazı değerler defalarca hesaplanmış olur.

5.3 Dinamik Programlama ile Çözüm

Hocaya göre aynı rekürans bağıntısına uygun hesaplamayı bir dizi üzerinde iteratif olarak yapsaydık bu tekrarlar olmazdı. İlk iki değer ($F(0) = 0$ ve $F(1) = 1$) yazılır. Sonraki her hücreye önceki iki hücrenin toplamı yazılır:

n	0	1	2	3	4	5	6	7	8
$F(n)$	0	1	1	2	3	5	8	13	21

Böylece bir hücre için baştan itibaren hesaplama yapmaya gerek kalmaz. Dinamik programlama mantığı, önceki işlemlerin sonucunu kullanarak ana sonuca ulaşmayı sağlar.

5.4 Dinamik Programlamanın Bedeli: Bellek

Hoca, ekrandaki yaklaşımın önemli bir işlem avantajı olduğunu, ama bir de sorunu olduğunu söyledi ve sorunun ne olduğunu sordu.

Soru: Dinamik programlamanın bu yaklaşımdaki en büyük sorunu nedir? **Cevap:** Bir öğrenci "sürekli diziye erişim" dedi. Hoca, özyinelemeli tarafta da fonksiyon çağrısı yapıldığını, başka bir şey aradığını söyledi. Başka bir öğrencinin cevabı doğruydu: çok bellek kaplayabilir.

> **Hocanın vurgusu:** Dinamik programlamanın genellikle bellek karmaşıklığı (space complexity) açısından bir dezavantajı vardır. Problemi çözmeden önce n'e ya da başka bir parametreye bağlı olarak ne kadar alan gerektiği hesaplanmalıdır. Genelde zaman karmaşıklığına (time complexity) odaklanırız. Ama dinamik programlamayla çözmeyi planladığımız problemlerde bellek karmaşıklığını da ortaya koyup, kullanılacak sayılara göre yöntemin ilgili cihazda kullanılmaya uygun olup olmadığını gözden geçirmek gerekir.
>
> **Ders sonrası not:** Derste özyinelemeli çözüm için yalnızca "çok fazla çağrı" dendi. Python ile sayıldığında $F(5)$ hesaplanırken $F(3)$ 2 kez, $F(2)$ 3 kez, $F(1)$ 5 kez, $F(0)$ 3 kez çağrılır (toplam 15 çağrı). Çağrı sayısı n ile üstel büyür: $\Theta(\phi^n)$, $\phi \approx 1{,}618$. Dizi ile çözüm $\Theta(n)$ zaman alır. Fibonacci'de yalnız son iki değer gerektiği için bellek $\Theta(1)$'e de indirilebilir.

6. Para Sırası Problemi (Coin-Row Problem)

Hoca, ilerideki örneklerden daha basit olan bir problemle devam etti.

6.1 Problemin Tanımı

Arka arkaya dizilmiş paralar (coin) verilir. Amaç, önümüzdeki paralardan en yüksek toplam miktarı seçmektir.

- **Kısıt:** Yan yana iki para seçilemez.
- **Hedef:** Seçilen paraların toplamı en büyük olmalıdır.

Örnek dizi:

$$c = 5,\ 1,\ 2,\ 10,\ 6,\ 2$$

6.2 Kaba Kuvvet Neden Uygun Değil?

Hoca önce olası seçimlerden örnekler verdi: 5, 2, 6 seçilebilir; 1, 10, 2 seçilebilir; 5, 10, 2 seçilebilir. Bütün bu kombinasyonları denemek gerekir. Örnekte 6 para var, ama normalde n para olur ve n genelde büyük bir sayıdır. Problem kombinatorik olduğu için bütün kombinasyonları hesaplamak çok maliyetli bir kaba kuvvet (brute force) yaklaşımı olur. Hepsini tek tek denememek için dinamik programlama kullanılır.

Bir öğrenci, 6 paranın 3'er 3'er ayrıldığını ve tek sayıda paralarda farklı, çift sayıda paralarda farklı bir seçim yapılması gerektiğini düşündü. Hoca seçimin 3-3 olmak zorunda olmadığını,

tek-çift ayrımına gerek olmadan problemin çözülebileceğini söyledi. Önemli olan, ilerlerken her noktada hangisini almaya karar vereceğimizdir.

6.3 Tabloyu Sözel Olarak Doldurmak

Hoca üç satırlı bir tablo çizdi: birinci satır indis, ikinci satır her paranın değeri (c), üçüncü satır o paraya kadar elde edilebilecek en büyük toplam (F).

Soru: Paralar 1'den başladığı hâlde neden 0 indisli bir sütun var? **Cevap:** Hoca, hiçbir para seçilmediğinde toplamın 0 olduğunu ve sıfırdan başlamak gerektiğini söyledi. Bunu bir sayaç (counter) gibi düşünmek gerekir.

Hoca tabloyu öğrencilerle adım adım doldurdu:

- **0. adım:** Hiç para yok. $F = 0$.
- **1. para:** Elimizde yalnız bir para olsaydı toplam 5 olurdu.
- **2. para:** Elimizde 5 ve 1 var. Yan yana oldukları için ikisi birden alınamaz. En yüksek olanı seçeriz: 5. Hoca burada "bir kere demek ki fonksiyonda en büyüğü seçme gibi bir şey olmalı" dedi.
- **3. para:** 1. ve 3. paralar birleştirilir: $5 + 2 = 7$.
- **4. para:** 1. ve 4. paralar birleştirilir: $5 + 10 = 15$.
- **5. para:** Toplam yine 15'tir.
- **6. para:** $2 + 15 = 17$.

5. parada hoca, 6'nın neden eklenemediğini sordu. İlk cevaplar "10 ile yan yana olduğu için" ve "1-4 arasındaki paraların en büyük toplamı zaten 15" yönündeydi. Hoca bunları kabul etmedi:

   Hocanın vurgusu: Şu anda siz gözle bakıp karar veriyorsunuz. Bilgisayarın böyle bir şansı yok.

Doğru akıl yürütmeyi bir öğrenci verdi: 6, 10'un hemen öncesine kadarki toplama, yani 7'ye eklenir. $6 + 7 = 13$ olur. Bu 15'ten küçük olduğu için 15 korunur. Hoca bunu şöyle özetledi: Yan yana olanları alamıyoruz. 10'u almayacaksak bir öncekini alma şansımız var. Yeni geleni 10'dan öncekilere ekleseydik 13 elde edecektik. Bu avantajlı olmadığı için 15'i tercih ettik.

Son adımda 2, 4. indiste bulunan 15'e eklenir ve 17 elde edilir. Hoca böylece bir formülün ortaya çıktığını söyledi: Ya o andaki parayı alıp öncekilerle işleme sokarız, ya da bir önceki durumdaki (state) değere bakıp onun daha büyük olup olmadığını kontrol ederiz.

6.4 Rekürans Bağıntısının Kurulması

Aradan sonra bir öğrenci mantığı şöyle anlattı: Dizinin uzunluğundan bir fazla boyutta yeni bir F dizisi tanımlanır, ilk elemanı 0'dır. $F(1)$ her zaman c_1 olur. $F(2)$ için önce c_2 ile $F(0)$ toplanır. Sonuç $F(1)$'den büyükse bu sonuç, küçükse bir önceki F değeri yazılır. Böyle devam edilir.

Öğrenci bunu formüle dökerken ilk olarak "$F(n) = c_n - c_{n-2}$, eğer c_{n-1}'den büyükse; değilse c_{n-1}" gibi bir ifade söyledi. Hoca eksi işaretini anlamadığını, öğrencinin anlatımının toplama dayandığını söyledi; doğrusu toplamadır. Öğrenci ayrıca bir yerde c yerine F yazılması gerektiğini fark etti. Hoca ifadeyi şöyle toparladı: c_{n-2} yerine $F(n-2)$ yazılır, çünkü oraya kadar olan **toplam** alınır. İkinci seçenek de c_{n-1} değil, $F(n-1)$, yani bir önceki paraya kadar gelmiş toplamdır.

$$F(n) = \max\{c_n + F(n-2),\ F(n-1)\}, \quad n > 1$$

$$F(0) = 0, \qquad F(1) = c_1$$

İki seçenek şunlardır:

- $c_n + F(n-2)$**:** O anki para seçilir. Bir önceki para alınamayacağı için $n-2$'ye kadar gelmiş toplama eklenir.
- $F(n-1)$**:** O anki para seçilmez. Bir önceki paraya kadar gelmiş toplam aynen alınır.

6.5 Formülle Tablonun Doldurulması

Hoca tabloyu bu kez formülle, öğrencilere söyleterek doldurdu:

- $F(0) = 0$, $F(1) = c_1 = 5$
- $F(2) = \max\{c_2 + F(0),\ F(1)\} = \max\{1 + 0,\ 5\} = 5$
- $F(3) = \max\{c_3 + F(1),\ F(2)\} = \max\{2 + 5,\ 5\} = 7$
- $F(4) = \max\{c_4 + F(2),\ F(3)\} = \max\{10 + 5,\ 7\} = 15$
- $F(5) = \max\{c_5 + F(3),\ F(4)\} = \max\{6 + 7,\ 15\} = 15$
- $F(6) = \max\{c_6 + F(4),\ F(5)\} = \max\{2 + 15,\ 15\} = 17$

İndis	0	1	2	3	4	5	6
c	-	5	1	2	10	6	2
F	0	5	5	7	15	15	17

Alınabilecek en büyük toplam 17'dir. Python ile bütün geçerli alt kümeler denendiğinde de en büyük toplam 17 çıkar.

6.6 Seçilen Paraların Geriye Doğru Bulunması

Hoca en büyük toplamın 17 olduğunu bildiğimizi, ama hangi paraların seçildiğini nasıl bulacağımızı sordu. Bir öğrenci gözle bakarak "1., 4. ve 6." dedi. Hoca bunun doğru olduğunu, ama bilgisayarın buna gözle bakmadan, yalnız tabloyu kullanarak nasıl karar vermesi gerektiğini sordu.

Öğrencilerle birlikte şu kural ortaya çıktı: Sondan başlanır ve her adımda yalnız yan yana iki F değeri karşılaştırılır.

- $F(i) \neq F(i-1)$ ise i. para alınmıştır. Bir önceki para alınamayacağı için doğrudan $i-2$'ye geçilir.
- $F(i) = F(i-1)$ ise i. para alınmamıştır. $i-1$'e geçilir.

Örnekte:

- $F(6) = 17 \neq F(5) = 15$: 6. para alındı. 5. para alınamaz, 4'e geçilir.
- $F(4) = 15 \neq F(3) = 7$: 4. para alındı. 3'ü atlayıp 2'ye geçilir.
- $F(2) = 5 = F(1) = 5$: 2. para alınmadı. 1'e geçilir.
- $F(1) = 5 \neq F(0) = 0$: 1. para alındı.

Seçilen paralar 1., 4. ve 6. paralardır: $5 + 10 + 2 = 17$. Hoca, böylece hem kurala uygun şekilde en fazla parayı aldığımızı, hem hangilerini aldığımızı bulduğumuzu, hem de kritik olan rekürans bağıntısını oluşturduğumuzu söyledi.

6.7 Karmaşıklık

Soru: Bu problemin zaman ve bellek karmaşıklığı nedir? **Cevap:** Bir öğrenci, dizi bir kez tarandığı için zamanın $O(n)$, aynı diziyi tuttuğumuz için belleğin de $O(n)$ olduğunu söyledi. Hoca bunlara $\Theta(n)$ da denebileceğini hatırlattı. Alt ve üst sınırlar aynı noktaya düştüğü için hem $\Omega(n)$ hem $O(n)$ geçerlidir, dolayısıyla $\Theta(n)$ yazılır.

$$T(n) = \Theta(n), \qquad S(n) = \Theta(n)$$

Hoca, n'de kaldığımız için bunun oldukça uygulanabilir (feasible) olduğunu, ama n büyük olsaydı $3 \times n$'lik bir tablo alanı gerektiğini söyledi. Örneğin bir milyon para olsaydı bir milyonluk alan açmak gerekirdi ve bu sıkıntı yaratabilirdi.

Soru: Yalnız en büyük toplam isteniyorsa bütün diziyi tutmak yerine yalnız önceki 3 elemanı tutan bir dizi kullanıp onu sürekli güncelleyemez miyiz? **Cevap:** Hoca bunun bir bakış açısı olduğunu ve değerlendirilebileceğini söyledi. Ama bu tür problemlerde genelde hangi eleman setinin kullanıldığı da arandığı için bütün tablo gerekir. Geriye doğru seçim bulma işlemi tablonun tamamına ihtiyaç duyar.

7. Sırt Çantası Problemi (0/1 Knapsack Problem)

Hoca, bu problemin daha önce farklı bir başlık altında dile getirildiğini hatırlattı.

7.1 Problemin Tanımı

Bir hırsız bir eve girer. Elinde bir çuval vardır. Çuvalın taşıyabileceği bir ağırlık kapasitesi vardır. Evdeki eşyaların (item) ağırlıkları ve değerleri bellidir. Hedef, bu ağırlık kapasitesini aşmadan evden toplam değeri en yüksek eşya kümesini çıkarmaktır.

- **Verilenler:** Toplam kapasite W; her eşyanın ağırlığı w_i ve değeri v_i.
- **Amaç:** Seçilen eşyaların toplam değerini en büyük yapmak.
- **Kısıt:** Seçilen eşyaların toplam ağırlığı W'yi aşmamalı.

7.2 Kaba Kuvvet Karmaşıklığı

Soru: Bu problemi kaba kuvvetle çözseydik karmaşıklık ne olurdu? **Cevap:** Bir öğrenci $O(n^2)$ dedi; hoca bunun doğru olmadığını söyledi. Başka bir öğrenci "iki üzeri n" dedi ve gerekçesini açıkladı: Tek eşyalı, iki eşyalı, üç eşyalı bütün kombinasyonlar denenir. Hoca bunu düzelterek boş küme hariç $2^n - 1$ kombinasyonun denenmesi gerektiğini, bunun oldukça yüksek bir sayı olduğunu söyledi.

7.3 İlk Örnek: Kombinasyonları Elle Saymak

Ağırlık sınırı $W = 5$ kilo olsun. Eşyalar:

Eşya	1	2	3	4	5
Ağırlık	2	1	3	4	5
Değer	10	20	15	50	90

Hoca sınırlar içinde alınabilecek kombinasyonları sordu. Öğrencilerle birlikte şunlar yazıldı:

Kombinasyon	Toplam ağırlık	Toplam değer
{1, 2}	3	30
{2, 3}	4	35
{1, 3}	5	25
{2, 4}	5	70
{5}	5	90

Bir öğrenci "2 ile 4, 6 ediyor, alamayız" diye itiraz etti. Hoca, 2 ve 4'ün eşya numaraları olduğunu, ağırlıklarının 1 ve 4 olduğunu, toplamın 5 ettiğini açıkladı. Öğrenci eşya numarası ile ağırlığı karıştırmıştı.

Bu şartlarda en avantajlısı tek başına 5. eşyayı almaktır (90). Hoca bunu "gözle hızlıca" yaptıklarını, ama bütün kombinasyonları denemeden bir çözüm üretmek gerektiğini söyledi.

> **Ders sonrası not:** Tabloda yalnız ikili kombinasyonlar ve 5. eşya yazıldı. Python ile kontrol edildiğinde tek eşyalı kombinasyonlar da ({1}, {2}, {3}, {4}) kapasiteye sığar. Bunların en büyüğü {4} = 50'dir. Sonuç değişmez: en iyi seçim {5} = 90'dır.

Soru: Ağırlık başına düşen değeri hesaplayıp ona göre bir algoritma hazırlamak doğru olur mu? **Cevap:** Hoca bunun bir bakış açısı olduğunu ve yeri geldiğinde başka bir algoritmik yaklaşımda kullanılacağını söyledi. Ama dinamik programlamada ağırlık başına değer hesaplanmaz. Para sırası problemindeki mantık uygulanır. Fark şudur: Para sırasında kısıt yan yana olmamaktı. Burada kısıt ağırlıktır. Bir eşya seçildiğinde toplam ağırlıkla ilişkilendirilmeli ve değeri hesaplanmalıdır. Yani iki parametre vardır: bir yanda ağırlık kısıtı, öbür yanda en büyük yapılmaya çalışılan değer.

Ek bilgi (derste anlatılmadı): Ağırlık başına değere göre seçim yapmak, açgözlü (greedy) yaklaşımdır. Eşyaların bölünebildiği kesirli sırt çantası (fractional knapsack) probleminde bu yaklaşım optimal sonucu verir. Eşyaların ya bütün olarak alındığı ya da hiç alınmadığı 0/1 sırt çantası probleminde ise her zaman optimal sonucu vermez. Hocanın "başka bir algoritmik yaklaşımda kullanacağız" dediği büyük olasılıkla açgözlü algoritmalardır.

7.4 Dinamik Programlama Tablosunun Yapısı

Hoca yeni bir örneğe geçti. Ağırlık sınırı yine $W = 5$'tir. Eşyalar:

Eşya	Ağırlık	Değer
1	2	12
2	1	10
3	3	20
4	2	15

Para sırasında, matrise benzese de bir dizi mantığı kullanılmıştı. Burada hoca bir **matris** kullanacağını söyledi: sütunlarda ağırlık, satırlarda eşyalar.

Soru: Bu matrisin kaç satırı ve kaç sütunu olur? **Cevap:** Bir öğrenci "4 satır, 4 sütun", başka biri "5 satır, 5 sütun" dedi. Hoca ikisine de "yaklaştın" dedi. Doğru cevap şudur: 4 eşya vardır. Ağırlıklar kesirli olmadığı için 1'er kilo ilerlenir, 5 ağırlık değeri vardır. Ama başlangıç değerlerine ihtiyaç olduğu için eşyalara bir satır, ağırlıklara bir sütun eklenir. Sonuç **5 satır, 6 sütundur**: eşya 0-4, ağırlık 0-5.

Genel olarak n eşya ve kapasite W için tablo $(n+1) \times (W+1)$ boyutundadır.

Hücrelerde ne tutulur? Ağırlıklar zaten sütunlarla takip edildiği için hücrelerde **değerler** tutulur. Hücre (i, w), ilk i eşya içinden en fazla w kilo ile alınabilecek en büyük değeri gösterir.

Soru: Sonuç matrisin hangi hücresinde oluşur? **Cevap:** Bir öğrenci, sol üstten başlandığı için sağ alt köşe olması gerektiğini söyledi. Hoca doğruladı. Burada en büyük değer elde edilir. Başka bir problemde belki en küçük değer elde edilirdi.

Soru: Matrisin hangi hücreleri sıfırlanmalı? **Cevap:** Bir öğrenci köşegen ve altını önerdi; hoca bunun doğru olmadığını söyledi. Doğru cevap **ilk satır ve ilk sütundur**. Hiç eşya yokken, kilo kısıtı ne olursa olsun değer 0'dır. Sıfır kiloda da hiçbir eşya alınamayacağı için her eşya sayısında değer 0'dır. Hoca fazladan bir satır ve bir sütunu tam da bu yüzden tanımladığını söyledi.

Soru: İlk satırı ve ilk sütunu zaten 0 olarak belirliyorsak neden kullanıyoruz? **Cevap:** Hoca bunun formül kurulduğunda daha net anlaşılacağını söyledi. Para sırasında $n = 2$ için $F(n-2)$ hesaplanırken bilgisayar gidip $F(0)$'ı topluyordu. Burada da formülün başlangıç değerlerine ihtiyacı vardır.

7.5 Tablonun Satır Satır Doldurulması

Hoca tabloyu öğrencilerle satır satır doldurdu. Bu aşamada formül henüz yazılmamıştı; değerler önceki satırlardan yararlanarak sözel olarak bulundu.

1. satır (yalnız 1. eşya: ağırlık 2, değer 12):

- 1 kilo: 0. İlk eşya 2 kilo olduğu için alınamaz.
- 2 kilo: 12.
- 3, 4, 5 kilo: Yine 12, çünkü elde yalnız bir eşya var.

2. satır (1. ve 2. eşyalar; 2. eşya: ağırlık 1, değer 10):

- 1 kilo: 10. 2. eşyanın ağırlığı 1, 1. eşyanınki 2 olduğu için yalnız 2. eşya alınabilir.
- 2 kilo: 12. İki eşya birlikte 3 kilo eder, sığmaz. Yalnız 1. eşya alınır.
- 3, 4, 5 kilo: 22. İkisi birden alınabilir: $12 + 10 = 22$.

Burada bir öğrenci 2 kiloluk hücrenin de 22 olması gerekmez mi diye sordu. Hoca, 2 kiloya kadar alınabildiğinde iki eşyadan yalnız 1. eşyanın alınabileceğini hatırlattı.

3. satır (3. eşya da eklendi: ağırlık 3, değer 20):

- 1 kilo: 10.
- 2 kilo: 12.
- 3 kilo: 22. Alternatif 20'dir (yalnız 3. eşya). 22 seçilir.
- 4 kilo: 30 ($20 + 10$, yani 3. ve 2. eşyalar). Alternatif 22'dir. 30 seçilir.
- 5 kilo: 32 ($20 + 12$, yani 3. ve 1. eşyalar).

Hoca burada, hep alt problem olarak ilerlediklerini vurguladı: Önce yalnız bir eşya alınabiliyordu, sonra iki, sonra üç. Her satırdaki değerler bir önceki satırın değerlerinden elde ediliyor. Bunun şimdilik yalnız görsel yapıldığını, birazdan formüle edileceğini söyledi.

4. satır (4 eşya birden; 4. eşya: ağırlık 2, değer 15):

- 1 kilo: 10.
- 2 kilo: 15. Elde 15 ve 12 var, büyük olan 15'tir.
- 3 kilo: 25 ($15 + 10$). Alternatif olarak 20 söylendi.
- 4 kilo: 30. Alternatif 27'dir ($15 + 12$).
- 5 kilo: Önce 35 ($20 + 15$, 3. ve 4. eşyalar) önerildi. Bir alternatif olarak 32 söylendi. Sonunda bir öğrenci 37'yi buldu: 1., 2. ve 4. eşyalar, $12 + 10 + 15 = 37$.

Hoca, bu gruptan 5 kilo sınırı içinde alınabilecek en yüksek değerin 37 olduğunu söyledi.

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

Tablo Python ile doğrulandı. Seçilen eşyalar 1, 2 ve 4'tür: toplam ağırlık $2 + 1 + 2 = 5$, toplam değer $12 + 10 + 15 = 37$.

> **Ders sonrası not:** 4. satırda 3 kiloluk hücre için "alternatif 20" söylendi. Formüle göre bu hücrede 4. eşyayı almamak, bir üst satırdaki $P(3, 3) = 22$ değerini verir. Doğru karşılaştırma $\max\{22,\ 15 + P(3, 1)\} = \max\{22,\ 25\} = 25$'tir. Sonuç (25) değişmez. Benzer şekilde 5 kiloluk hücrede alınmama seçeneği $P(3, 5) = 32$, alınma seçeneği $15 + P(3, 3) = 37$'dir. Sözü geçen 35 ise geçerli bir kombinasyondur ({3, 4}), ama formüldeki iki seçenekten biri değildir.

7.6 Formüle Doğru

Hoca bunu gözle yaptıklarını, önceki bilgilerden yararlandıklarını, ama henüz formüle etmediklerini söyledi. Önce ne yapmak istendiği yazıldı:

$$\text{maximize} \sum v_i \quad \text{subject to} \quad \sum w_i \leq W$$

Hoca, eşyalar üzerinde ilerlerken her eşya için kaç seçenek olduğunu sordu. İki seçenek vardır:

- **Durum 1:** i. eşyayı al (take item i).
- **Durum 2:** i. eşyayı alma (not take item i).

Para sırasındaki F'ye karşılık burada bir P fonksiyonu tanımlandı. Hoca P'yi "price" sözcüğüyle ilişkilendirdi.

Soru: P'nin iki parametresi neyi temsil etmeli? **Cevap:** Bir öğrenci "ağırlık ve eşya" dedi. Hoca sırayı eşya, ağırlık olarak koydu: $P(i, w)$.

Formülü bulmak için hoca 37 değerine nasıl ulaşıldığını sordu: Bu hücreye gelirken hangi hücreler kullanılabilir, üstündeki mi, yanındaki mi, sol çaprazındaki mi? Bir öğrenci, aynı eşya sayısında daha az ağırlıklı hücrelerin ve daha az eşyayla daha az ağırlıkta alınabilecek hücrelerin, yani "sol üst taraftaki" değerlerin kullanılabileceğini söyledi.

Hoca somut bir hücre seçti: $P(3,3) = 22$. Bu değere 3. eşyayı alarak mı, başka türlü mü ulaşıldığını sordu. Ardından "$P(2,1)+v_3$ yazsam olur mu?" diye sordu ve bir öğrenci evet dedi. Hoca burada $P(2,1)$'in 2. satırın 1 kiloya denk gelen hücresi olduğunu, v_3'ün de 3. eşyanın değeri olduğunu açıkladı.

> **Ders sonrası not:** $P(2,1)+v_3 = 10+20 = 30$'dur. Bu, $P(3,3)$'ün değil, $P(3,4)$'ün "alma" seçeneğidir. 3. eşyanın ağırlığı 3 olduğu için $P(3,3)$'te alma seçeneği $v_3 + P(2,3-3) = 20 + P(2,0) = 20$'dir. Bu, alınmama seçeneği $P(2,3) = 22$ ile karşılaştırılır ve $P(3,3) = 22$ bulunur.

Formül hemen kurulamayınca hoca daha basit bir hücreden başladı: $P(1,3)$.

$$P(1,3) = \max\{P(0,1) + 12,\ P(0,3)\} = \max\{0+12,\ 0\} = 12$$

Burada $P(0,1)$, 1. eşya (ağırlık 2) alındığında geriye kalan $3-2=1$ kilo için bir üst satırdaki değerdir. 12, 1. eşyanın değeridir. $P(0,3)$ ise 1. eşyayı almama seçeneğidir.

Bir öğrenci bu açıklamada v_3'ün neyi temsil ettiğini anlamadığını söyledi. Hoca bunun 3. eşyanın değeri (value) olduğunu, $P(1,3)$ hesabındaki 12'nin de aynı rolü oynadığını gösterdi: Eşyayı almak ya da almamak hesaba katılırken alınan eşyanın değeri formüle eklenir.

Hoca genel $P(i,w)$ formülünü öğrencilere bıraktı ve saat 15:00'teki uygulama oturumunda birlikte yazacaklarını söyledi. Kayıt formül yazılmadan biter.

7.7 Genel Formül ve Karmaşıklık

Derste kurulmaya çalışılan ve $P(1,3)$ örneğinin genelleştirilmesiyle elde edilen bağıntı şudur:

$$P(i,w) = \begin{cases} \max\{P(i-1,w),\ v_i + P(i-1,w-w_i)\} & w_i \le w \\ P(i-1,w) & w_i > w \end{cases}$$

$$P(0,w) = 0, \qquad P(i,0) = 0$$

Bu formül derste tam olarak yazılmadı; çözüm aşağıdaki Örnek Sorular bölümünde verilmiştir. Tablo $(n+1)(W+1)$ hücredir ve her hücre sabit sürede doldurulur. Zaman ve bellek $\Theta(nW)$'dir.

> **Ek bilgi (derste anlatılmadı):** $\Theta(nW)$ karmaşıklığı sözde polinom (pseudo-polynomial) olarak adlandırılır. Çünkü W girdinin boyutu değil, bir sayının değeridir. W çok büyükse tablo da çok büyür. Hocanın "dinamik programlamada bellek ihtiyacını önceden hesaplayın" uyarısı bu problemde özellikle önemlidir.

8. Örnek Sorular

Bu derste kare ya da video yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:24:41): Bir sınıflandırma probleminde n özellik var. En yüksek başarıyı sağlayan özellik alt kümesini bulmak için kaç farklı seçeneğe bakmak gerekir?

Çözüm (derste verildi): $2^n - 1$. Her özellik ya seçilir ya seçilmez (2^n). Boş küme, yani hiç özellik kullanmamak, geçerli bir seçenek olmadığı için çıkarılır.

Soru 2 (derste sözlü, 00:21:04): Bir optimizasyon problemi örneği veriniz.

Çözüm (derste verildi): Grafı en az renkle boyama, otobüs güzergâhı optimizasyonu, ders programı yerleştirme, belli bir bütçeyle en fazla katkıyı sağlayacak malzemeleri alma. "Bir algoritmayı hızlandırmak" bir optimizasyon problemi değildir.

Soru 3 (derste sözlü, 00:34:23): Fibonacci sayılarının özyinelemeli çözümünün sıkıntısı nedir?

Çözüm (derste verildi): Aynı alt problemler tekrar tekrar hesaplanır. Örneğin $F(n-1)$ hesaplanırken $F(n-2)$ yeniden hesaplanır.

Soru 4 (derste sözlü, 00:37:16): Fibonacci'yi dizi ile hesaplayan dinamik programlama yaklaşımının dezavantajı nedir?

Çözüm (derste verildi): Bellek kullanımı. Sonuçları saklamak için ek alan gerekir. Problemin başında ne kadar alan gerektiği hesaplanmalıdır.

Soru 5 (derste sözlü, 00:39:58): $c = 5, 1, 2, 10, 6, 2$ dizisinde yan yana iki para seçmeden alınabilecek en büyük toplam için rekürans bağıntısı nedir? Tabloyu doldurunuz.

Çözüm (derste verildi): $F(n) = \max\{c_n + F(n-2),\ F(n-1)\}$, $F(0) = 0$, $F(1) = c_1$. Tablo: $F = 0, 5, 5, 7, 15, 15, 17$. En büyük toplam 17'dir.

Soru 6 (derste sözlü, 01:10:07): Aynı örnekte hangi paralar seçildi? Bilgisayar buna gözle bakmadan nasıl karar verir?

Çözüm (derste verildi): Sondan başlanır. $F(i) \neq F(i-1)$ ise i. para alınmıştır ve $i-2$'ye geçilir; eşitse alınmamıştır ve $i-1$'e geçilir. Seçilenler 1., 4. ve 6. paralardır ($5+10+2=17$).

Soru 7 (derste sözlü, 01:13:44): Para sırası probleminin zaman ve bellek karmaşıklığı nedir?

Çözüm (derste verildi): İkisi de $\Theta(n)$'dir. Dizi bir kez taranır ve n uzunluğunda bir tablo tutulur.

Soru 8 (derste sözlü, 01:17:18): Sırt çantası problemini kaba kuvvetle çözmenin karmaşıklığı nedir?

Çözüm (derste verildi): Boş küme hariç bütün alt kümeler denenir: $2^n - 1$ kombinasyon.

Soru 9 (derste sözlü, 01:19:18): $W = 5$, ağırlıklar $2, 1, 3, 4, 5$, değerler $10, 20, 15, 50, 90$ iken kapasiteye sığan kombinasyonlar hangileridir, en iyisi hangisidir?

Çözüm (derste verildi): {1, 2} = 30, {2, 3} = 35, {1, 3} = 25, {2, 4} = 70, {5} = 90. En iyisi {5} = 90'dır. (Tek eşyalı kombinasyonlar da sığar, sonuç değişmez.)

Soru 10 (derste sözlü, 01:23:49): 4 eşya ve $W = 5$ için dinamik programlama matrisi kaç satır ve kaç sütundur? Hangi hücreler sıfırlanır? Sonuç hangi hücrededir?

Çözüm (derste verildi): 5 satır (eşya 0-4), 6 sütun (ağırlık 0-5). İlk satır ve ilk sütun 0'dır. Sonuç sağ alt köşededir.

Soru 11 (derste sözlü, 01:28:03): Ağırlıklar $2, 1, 3, 2$, değerler $12, 10, 20, 15$, $W = 5$ için tabloyu doldurunuz.

Çözüm (derste verildi): Tablo 7.5'teki gibidir. Sağ alt köşe 37'dir; seçilen eşyalar 1, 2 ve 4'tür.

Soru 12 (derste sözlü, 01:41:51): $P(1,3) = \max\{P(0,1) + 12,\ P(0,3)\}$ mantığını genelleştirerek $P(i,w)$ için rekürans bağıntısını yazınız.

> **Çözüm (derste verilmedi):** i. eşya için iki seçenek vardır. Alınmazsa değer $P(i-1,w)$ olur. Alınırsa (yalnız $w_i \leq w$ ise) değer $v_i + P(i-1, w-w_i)$ olur. Buna göre $P(i,w) = \max\{P(i-1,w),\ v_i + P(i-1,w-w_i)\}$ ($w_i \leq w$), $P(i,w) = P(i-1,w)$ ($w_i > w$); $P(0,w) = P(i,0) = 0$. Bu bağıntı Python ile uygulandığında Soru 11'deki tablonun bütün hücreleri aynen elde edilir.

Dersin Toparlaması

Bu derste dinamik programlamanın temel fikri görüldü: Örtüşen alt problemler bir kez çözülür, sonuçları bir dizi ya da tabloda saklanır ve büyük problemin çözümüne bu değerlerden ulaşılır. Yöntemin dayanağı optimallik ilkesidir. Fibonacci örneğinde özyinelemeli çözümün aynı değerleri tekrar tekrar hesapladığı, dizi ile çözümün bunu önlediği, ama bunun bedelinin bellek olduğu gösterildi. Para sırası probleminde her adımda "al ya da alma" seçimi bir rekürans bağıntısına dönüştürüldü, tablo dolduruldu ve seçilen paralar tablodan geriye doğru bulundu.

Sırt çantası probleminde aynı “al ya da alma” mantığı iki parametreli bir tabloya taşındı: satırlar eşyalar, sütunlar ağırlık sınırı. Tablo öğrencilerle satır satır dolduruldu ve cevap sağ alt köşede 37 olarak bulundu. Genel $P(i, w)$ formülü derste tamamlanmadı; hoca bunu saat 15:00’teki uygulama oturumunda birlikte yazacaklarını ve bir örnek daha çözeceklerini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.